

.NET Dependency Injection (DI)

Introduction

Dependency Injection (DI) is one of the most fundamental design patterns in modern software development. It is heavily used in ASP.NET Core and .NET applications to create loosely coupled, maintainable, testable, and scalable software.

Instead of allowing a class to create its own dependencies, Dependency Injection provides those dependencies from an external source. This simple concept significantly improves application architecture.

Dependency Injection became a built-in feature starting with **ASP.NET Core** and continues to be the standard approach in **.NET 5, .NET 6, .NET 7, .NET 8, and newer versions**.

Why Dependency Injection?

Consider the following example.

```
public class EmailService
{
    public void SendEmail(string message)
    {
        Console.WriteLine(message);
    }
}

public class OrderService
{
    private EmailService _emailService = new EmailService();

    public void PlaceOrder()
    {
        // Business Logic

        _emailService.SendEmail("Order Placed");
    }
}
```

Although this works, several problems exist.

- OrderService directly creates EmailService.
- Tight coupling exists between the classes.

- EmailService cannot be easily replaced.
- Unit testing becomes difficult.
- Violates the Dependency Inversion Principle (SOLID).

Now consider Dependency Injection.

```
public class OrderService
{
    private readonly EmailService _emailService;

    public OrderService(EmailService emailService)
    {
        _emailService = emailService;
    }

    public void PlaceOrder()
    {
        _emailService.SendEmail("Order Placed");
    }
}
```

The OrderService no longer creates EmailService.

Instead, someone else supplies it.

This is the core idea of Dependency Injection.

What is a Dependency?

A dependency is simply another object that a class needs in order to perform its work.

Example:

```
graph TD
    OrderService --> EmailService
```

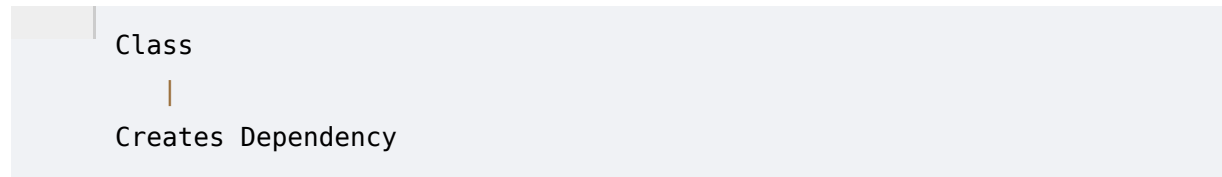
OrderService depends on EmailService.

Dependency Injection removes the responsibility of creating EmailService from OrderService.

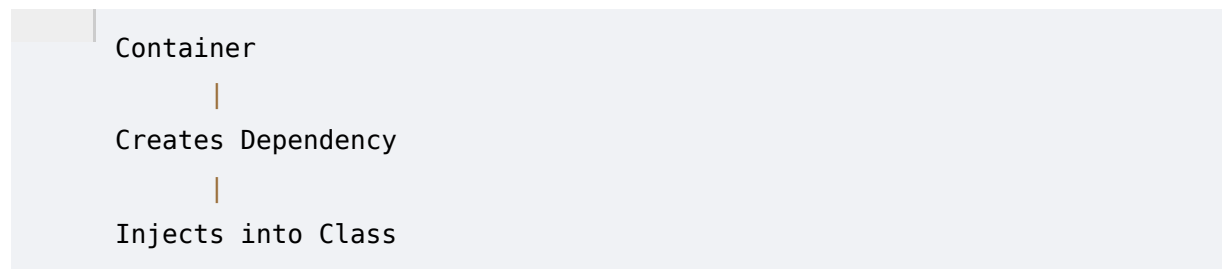
What is Inversion of Control (IoC)?

Dependency Injection is an implementation of a broader principle called **Inversion of Control (IoC)**.

Normally:



With IoC:



Instead of classes controlling object creation, the framework controls object creation.

Dependency Injection Container

.NET provides a built-in Dependency Injection Container.

Its responsibilities include:

- Creating objects
- Managing object lifetime
- Resolving dependencies
- Injecting required services

In ASP.NET Core, services are registered inside **Program.cs**.

Example:

```
builder.Services.AddScoped<IEmailService, EmailService>();
```

Later, whenever an `IEmailService` is required, the container automatically provides an `EmailService` instance.

Constructor Injection

Constructor Injection is the most common and recommended approach.

Example:

```
public interface IMessageService
{
    void Send(string message);
}

public class EmailService : IMessageService
{
    public void Send(string message)
    {
        Console.WriteLine(message);
    }
}

public class NotificationService
{
    private readonly IMessageService _messageService;

    public NotificationService(IMessageService messageService)
```

Advantages:

- Dependencies are explicit.
- Easier testing.
- Immutable dependencies.
- Encourages loose coupling.

Registering Services

Services are registered inside Program.cs.

Example:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IMessageService, EmailService>();

builder.Services.AddScoped<NotificationService>();

var app = builder.Build();
```

Once registered, .NET automatically resolves them.

Resolving Dependencies

Example Controller:

```
public class HomeController : Controller
{
    private readonly IMessageService _service;

    public HomeController(IMessageService service)
    {
        _service = service;
    }

    public IActionResult Index()
    {
        _service.Send("Welcome");

        return View();
    }
}
```

No object creation occurs inside the controller.

Everything is injected.

Service Lifetimes

One of the most important concepts in Dependency Injection is object lifetime.

.NET supports three lifetimes.

1. Transient

Creates a new object every time it is requested.

Registration:

```
builder.Services.AddTransient<IMessageService, EmailService>();
```

Characteristics:

- New instance every injection
- Lightweight services
- Stateless services

Example:

Request

Service A ----> New Instance

Service B ----> Another New Instance

2. Scoped

Creates one instance per HTTP request.

Registration:

```
builder.Services.AddScoped<IMessageService, EmailService>();
```

Characteristics:

- Shared throughout one request
- New request creates new object
- Most commonly used

Example:

Request 1

Controller

|

Service

|

Repository

(All share same instance)

Request 2

Entirely new instance

3. Singleton

Creates only one instance for the entire application.

Registration:

```
builder.Services.AddSingleton<IMessageService, EmailService>();
```

Characteristics:

- One instance
- Shared across all users
- Thread-safe implementation required

Example:

```
graph TD
    User1 --- Service[Same Service Instance]
    User2 --- Service
    User3 --- Service
```

Entire Application

User1
User2
User3

Same Service Instance

Lifetime Comparison

Lifetime	Created	Shared	Typical Usage
Transient	Every request	No	Lightweight services
Scoped	Once per HTTP request	Within request	Business services
Singleton	Once per application	Entire application	Configuration, Cache

Dependency Chain

The container resolves dependencies recursively.

Example:

```
Controller
  |
OrderService
  |
Repository
  |
DbContext
```

.NET automatically creates every required object.

Interface-Based Injection

Best practice is to inject interfaces rather than concrete classes.

Example:

```
public interface ILoggerService
{
    void Log(string message);
}

public class FileLogger : ILoggerService
{
    public void Log(string message)
    {
    }
}

builder.Services.AddScoped<ILoggerService, FileLogger>();
```

Benefits:

- Loose coupling
 - Easier testing
 - Easy replacement
-

Multiple Dependencies

A constructor may have multiple dependencies.


```
public class OrderService
{
    public OrderService(
        IRepository repository,
        ILoggerService logger,
        IEmailService email)
    {
    }
}
```

.NET resolves every dependency automatically.

Built-in Framework Services

Many services are already registered.

Examples include:

```
ILogger<T>

IConfiguration

IWebHostEnvironment

IHttpContextAccessor
```

These can be injected directly.

Example:

```
public class HomeController
{
    public HomeController(
        ILogger<HomeController> logger,
        IConfiguration configuration)
    {
    }
}
```

Dependency Injection in Repository Pattern

Example:

```
Controller
  |
Service
  |
Repository
  |
DbContext
```

Registration:

```
builder.Services.AddScoped<IProductRepository, ProductRepository>();

builder.Services.AddScoped<IProductService, ProductService>();

builder.Services.AddDbContext<AppDbContext>();
```

Unit Testing with Dependency Injection

Without DI:

Testing becomes difficult.

With DI:

```
var mockEmailService = new Mock<IEmailService>();

var service = new OrderService(mockEmailService.Object);
```

Mock objects replace real implementations.

Benefits:

- Faster tests
- No database
- No API calls
- Better isolation

Common Mistakes

Injecting Concrete Classes

Bad:

```
public OrderService(EmailService email)
```

Good:

```
public OrderService(IEmailService email)
```

Registering Wrong Lifetime

Example:

Registering DbContext as Singleton.

Incorrect:

```
builder.Services.AddSingleton<AppDbContext>();
```

Correct:

```
builder.Services.AddDbContext<AppDbContext>();
```

Service Locator Pattern

Avoid:

```
var service = serviceProvider.GetService<IEmailService>();
```

Prefer constructor injection.

Too Many Dependencies

Example:

```
public OrderService(  
    A,  
    B,  
    C,  
    D,  
    E,  
    F,  
    G,  
    H)
```

This usually indicates that the class has too many responsibilities and should be refactored.

Best Practices

- Prefer constructor injection.
 - Inject interfaces instead of concrete classes.
 - Keep services small and focused.
 - Choose appropriate lifetimes.
 - Avoid static services when DI can be used.
 - Keep dependencies explicit.
 - Follow SOLID principles.
 - Register services in one place.
 - Use Scoped for database-related services.
 - Make Singleton services thread-safe.
-

Advantages of Dependency Injection

- Loose coupling
 - Easier maintenance
 - Better testing
 - Better scalability
 - Improved readability
 - Easier mocking
 - Better separation of concerns
 - Cleaner architecture
 - Supports SOLID principles
 - Framework-managed object creation
-

Disadvantages

Although DI offers numerous benefits, improper use can introduce some complexity.

Potential disadvantages include:

- Additional learning curve for beginners.
- Large dependency graphs can be harder to understand.
- Incorrect service lifetime configuration can lead to runtime issues.
- Overusing interfaces for trivial classes may introduce unnecessary abstraction.

These disadvantages are generally outweighed by the long-term maintainability and flexibility that DI provides.

Real-World Example in ASP.NET Core

Consider a typical e-commerce application:



Service registrations:

```
builder.Services.AddScoped<IProductRepository, ProductRepository>();  
builder.Services.AddScoped<IProductService, ProductService>();  
builder.Services.AddDbContext<AppDbContext>();
```

When a request reaches `ProductController`, the DI container automatically creates and injects all required dependencies. Developers do not need to manually instantiate any of these objects, resulting in cleaner, more maintainable code.

Summary

Dependency Injection is a cornerstone of modern .NET application development. By delegating object creation to the built-in DI container, applications become loosely coupled, easier to test, and simpler to maintain. Understanding service lifetimes (Transient, Scoped, and Singleton), constructor injection, interface-based

programming, and proper registration practices is essential for building robust ASP.NET Core applications. Mastering DI also lays the foundation for advanced architectural patterns such as Repository, Unit of Work, CQRS, and Microservices, making it an indispensable skill for professional .NET developers.